

本讲内容： 分类

概览

- 什么是分类？
- 分类 vs. 回归
- 分类的过程
 - 建立模型阶段
 - 使用模型阶段
- 分类过程用到的三类数据集
- 常用的分类方法

什么是分类

- 属于模型挖掘，更确切地讲，是预测建模的过程
- 预测建模的目的是，根据观察到的对象特征值预测它的其他特征值
- 例如，根据年龄、收入、是否学生、信用情况这四个方面的特征，预测某个人是否会购买笔记本电脑
- 这是一个典型的分类问题

分类 vs. 回归

- 分类

- 构造、使用模型来对某个样本的类别进行判别
- 主要用于对离散的数据进行预测
- 典型应用：信誉评估、医学诊断

- 回归

- 构造、使用模型来对某个样本的值进行估计，例如预测某个不知道的值或者缺失值
- 主要用于对连续或有序的数据进行预测
- 典型应用：性能预测

分类的过程

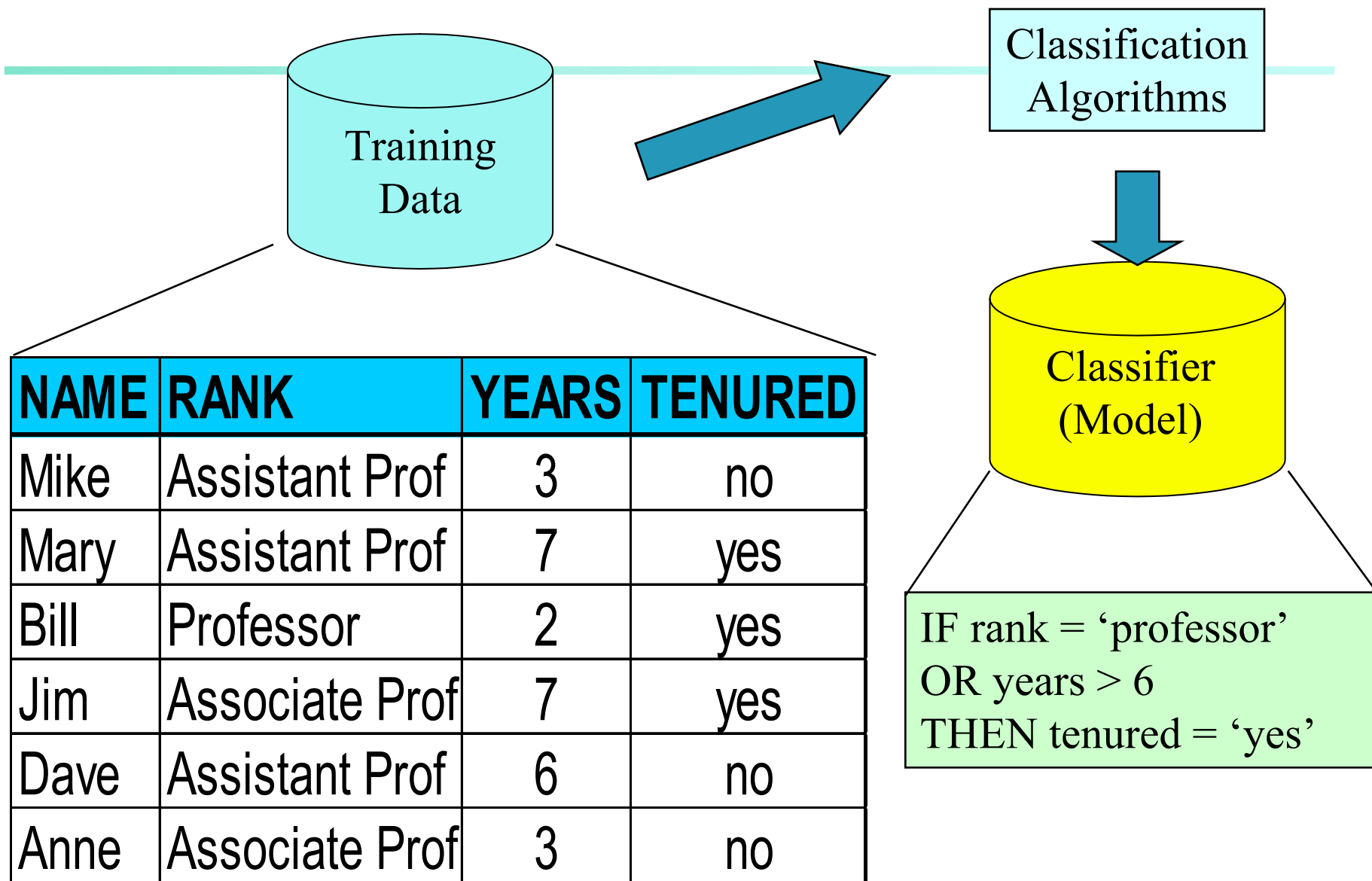
- 第一步，建立模型阶段

- 用来构造模型的数据集被称为训练集
- 模型一般表示为：分类规则，决策树或者数学公式

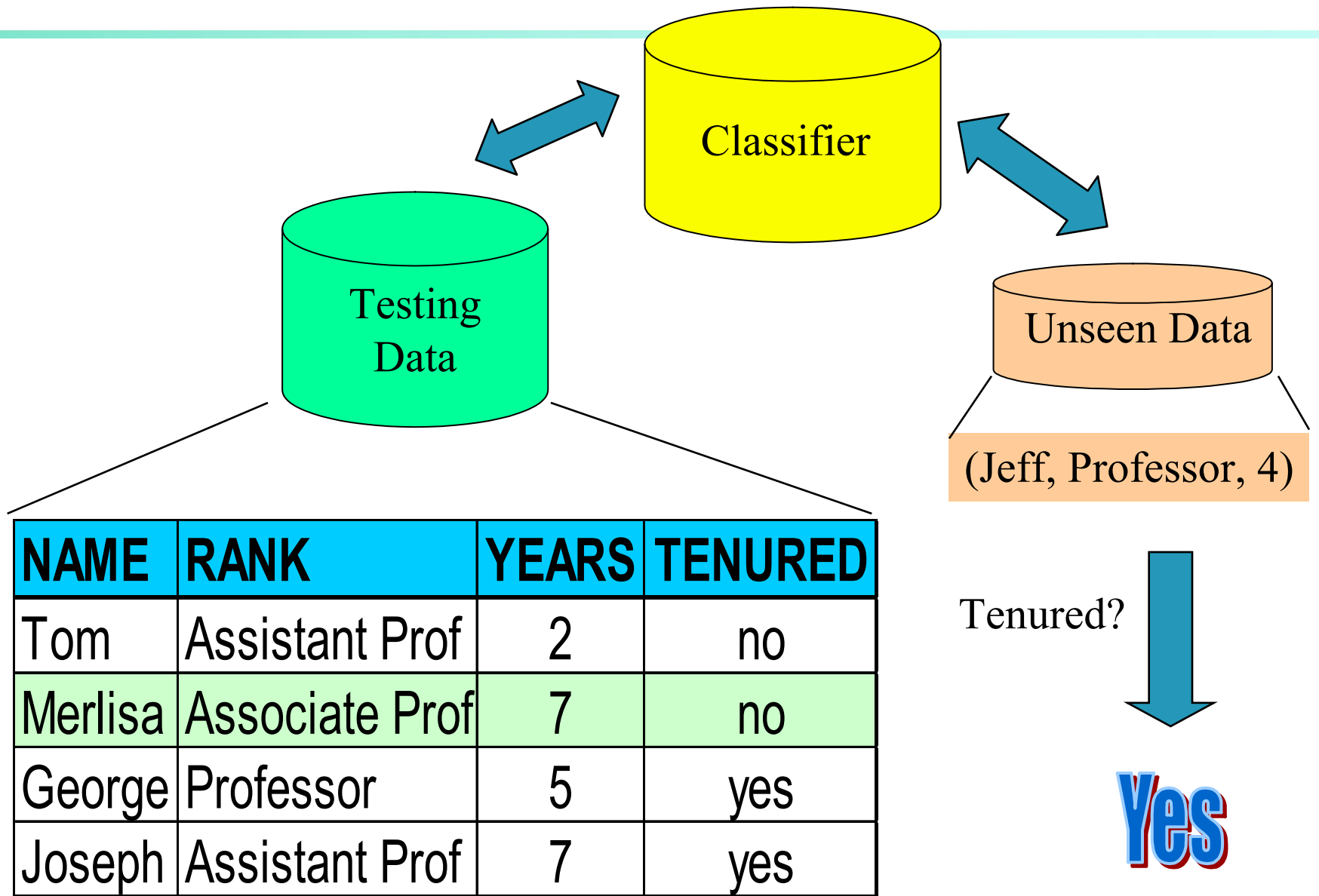
- 第二步，使用模型阶段

- 首先测试模型的准确性
 - 用测试集和由模型进行分类的结果进行比较
 - 两个结果相同所占的比率称为准确率
 - 测试集和训练集必须不相关
- 如果准确性可以接受的话，使用模型对新数据进行分类

分类过程：建立模型阶段

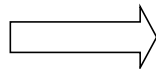


分类过程：使用模型阶段



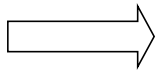
分类过程用到的三类数据集

训练集



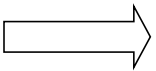
NAME	RANK	YEARS	TENURED
Mike	Assistant Prof	3	no
Mary	Assistant Prof	7	yes
Bill	Professor	2	yes
Jim	Associate Prof	7	yes
Dave	Assistant Prof	6	no
Anne	Associate Prof	3	no

测试集



NAME	RANK	YEARS	TENURED
Tom	Assistant Prof	2	no
Merlisa	Associate Prof	7	no
George	Professor	5	yes
Joseph	Assistant Prof	7	yes

新数据



NAME	RANK	YEARS	TENURED
Jeff	Professor	4	?

Supervised vs. Unsupervised Learning

- Supervised learning (classification)
 - Supervision: The training data (observations, measurements, etc.) are accompanied by labels indicating the class of the observations
 - New data is classified based on the training set
- Unsupervised learning (clustering)
 - The class labels of training data is unknown
 - Given a set of measurements, observations, etc. with the aim of establishing the existence of classes or clusters in the data

简单例子

■ 分类

- 两岁宝宝，给他看几个水果，并告诉他：红的圆的是苹果，橘黄的圆的是橘子（建模型）
- 拿一个水果问宝宝：这个水果，红的圆的，是什么？（用模型）

■ 聚类

- 两岁宝宝，给他一堆水果，告诉他：根据颜色分成两堆。宝宝会将苹果分成一堆，橘子分成一堆。假如告诉他：根据大小分成3堆，则宝宝会根据大小分成3堆，苹果和橘子可能会放在一起。

分类

特征1	特征2	...	特征n	标签
✓	✓	✓	✓	?
✓	✓	✓	✓	?
✓	✓	✓	✓	?
...	...	...	...	...
✓	✓	✓	✓	?

聚类

特征1	特征2	...	特征n
✓	✓	✓	✓
✓	✓	✓	✓
✓	✓	✓	✓
...	...	...	...
✓	✓	✓	✓

数据预处理

- 数据清洗
 - 对数据进行预处理，去掉噪声，对缺失数据进行处理（用某个最常用的值代替或者根据统计用某个最可能的值代替）
- 相关分析（特征选择）
 - 去掉某些不相关的或者冗余的属性（如银行进行贷款信誉评估时不需要考虑属于一周的某一天）
- 数据转换
 - 对数据进行概括（如将连续的值离散成若干个区域，将街道等上升到城市）
 - 对数据进行规范化，将某个属性的值缩小到某个指定的范围之内

分类方法评价

- 准确性
- 速度
 - 构造模型的时间 (训练时间)
 - 使用模型的时间 (预测时间)
- 鲁棒性
 - 能够处理噪声和缺失数据
- 可伸缩性
 - 对磁盘级的数据库有效
- 易交互性
 - 模型容易理解，具有较好的洞察力

常用的分类方法

- 决策树分类
- 贝叶斯分类
- 支持向量机
- 神经网络
- K近邻分类

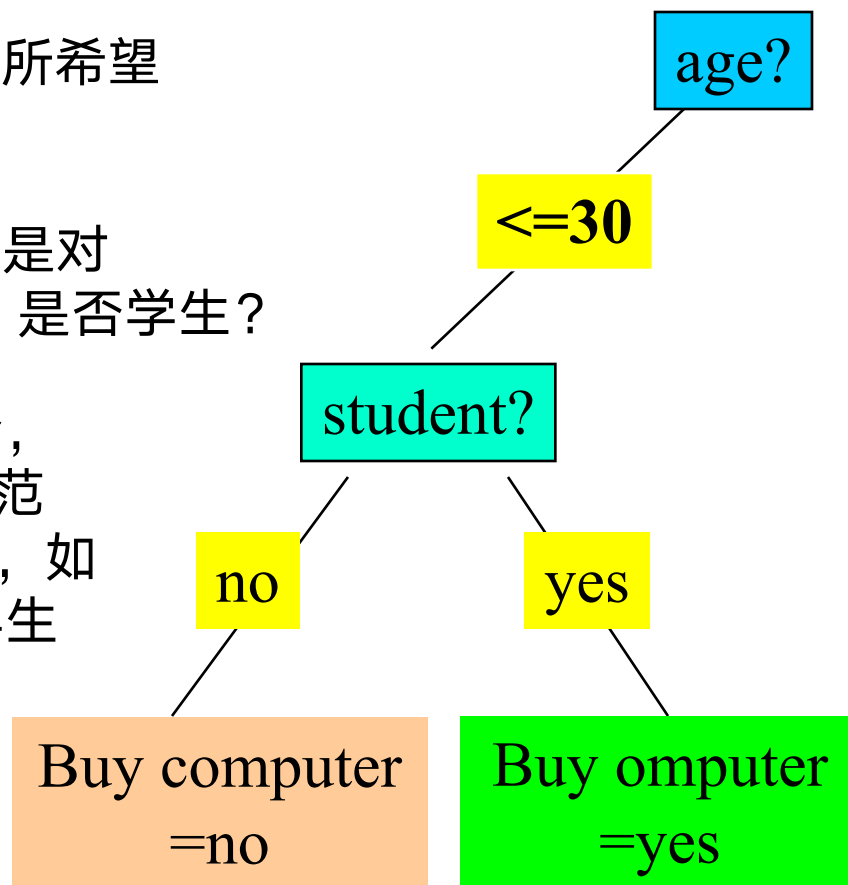
本讲内容

- 决策树分类

决策树模型

顾名思义，决策树是基于“树”结构来进行决策的，是人类在面临决策问题时一种很自然的处理机制。决策过程如下图所示：

- ❑ 决策过程中的最终结论对应了我们所希望的判定结果，如买或不买电脑
- ❑ 决策过程中提出的每个判定问题都是对某个属性的测试，如年龄是否 ≤ 30 ？是否学生？
- ❑ 每个测试的结果或是导出最终结论，或是导出进一步的判定问题，其考虑范围是在上次决策结果的限定范围之内，如在小于等于30岁的人中再看是否是学生



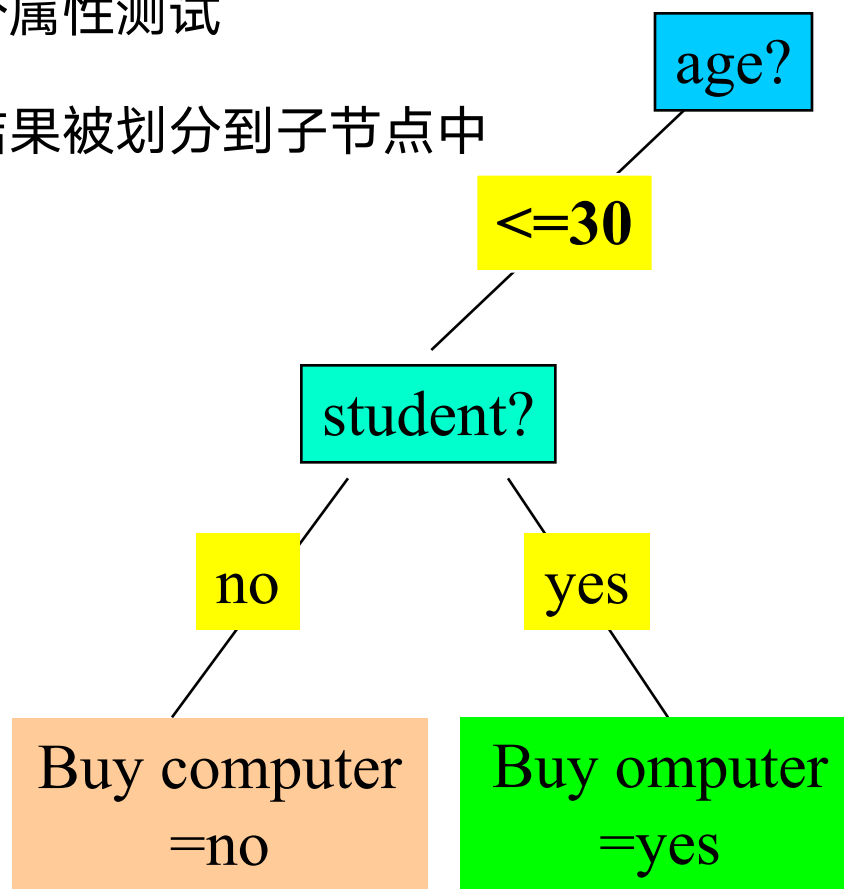
决策树模型

一般地，一颗决策树包含一个根节点、若干个内部节点和若干个叶子节点

- 叶子节点对应决策结果，其他节点对应一个属性测试
- 每个节点包含的样本集合根据属性测试的结果被划分到子节点中
- 根节点包含全部样本集

学习过程：通过对训练样本的分析来确定“划分属性”（即内部结点所对应的属性）

预测过程：将测试示例从根结点开始，沿着划分属性所构成的“判定测试序列”下行，直到叶子结点



决策树简史

- 第一个决策树算法：CLS (Concept Learning System)

[E. B. Hunt, J. Marin, and P. T. Stone's book "*Experiments in Induction*" published by Academic Press in 1966]

- 使决策树受到关注、成为机器学习主流技术的算法：ID3

[J. R. Quinlan's paper in a book "*Expert Systems in the Micro Electronic Age*" edited by D. Michie, published by Edinburgh University Press in 1979]

- 最常用的决策树算法：C4.5

[J. R. Quinlan's book "*C4.5: Programs for Machine Learning*" published by Morgan Kaufmann in 1993]

决策树简史(con't)

- 可以用于回归任务的决策树算法：CART (Classification and Regression Tree)

[L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone's book "Classification and Regression Trees" published by Wadsworth in 1984]

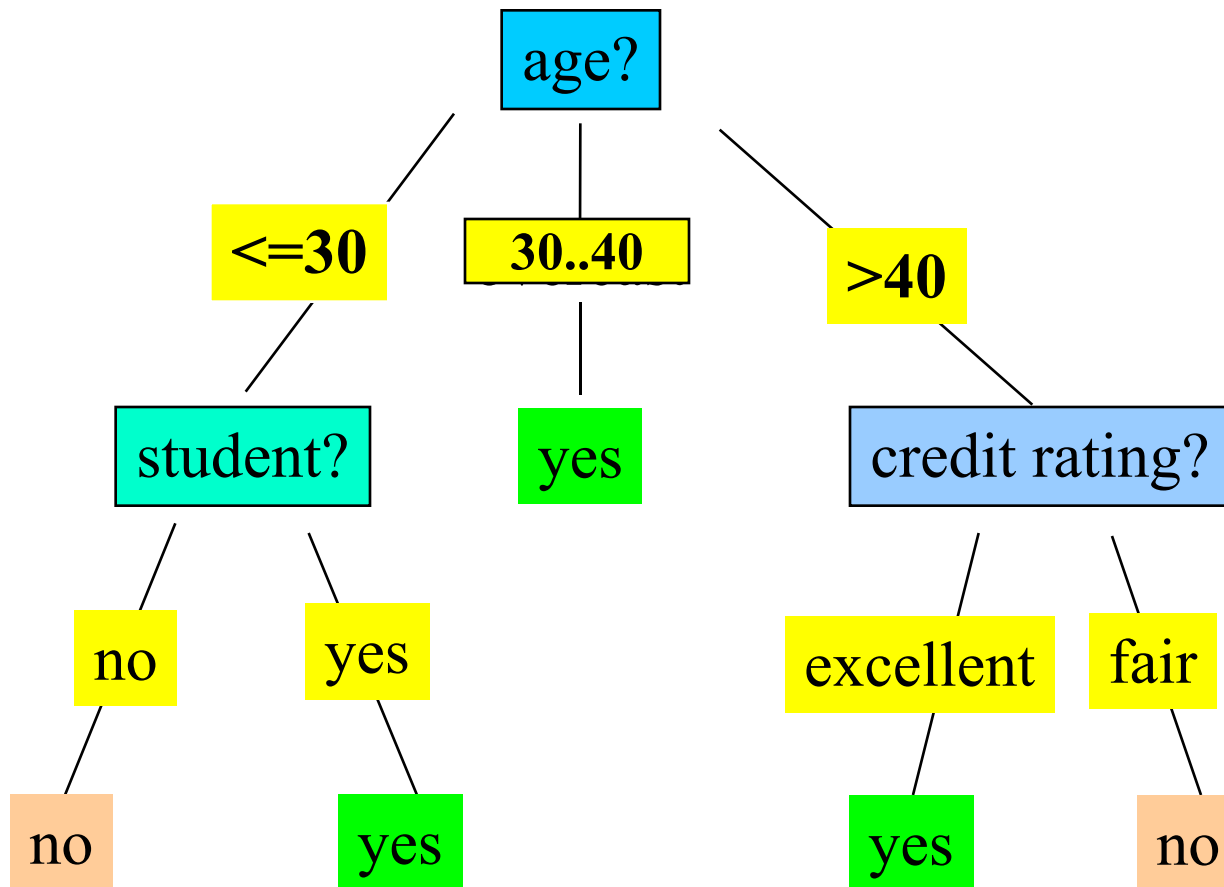
- 基于决策树的强大算法：RF (Random Forest)

[L. Breiman's MLJ'01 paper "Random Forest"]

Decision Tree Induction: Training Dataset

age	income	student	credit rating	buys computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

Output: A Decision Tree for “*buys_computer*”



决策树算法的宏观思考

- 最大高度 = 决策属性的个数
- 树越矮越好
- 要把重要的好的属性放在树根

因此，决策树建树算法就是：选择树根的过程

决策树分类

- 1 开始时，所有的训练集样本都在树根
- 2 属性都是可分类的属性(如果是连续值的话，先要对其进行离散化)

■ 根据训练集，划分训练集

停止划分的条件：

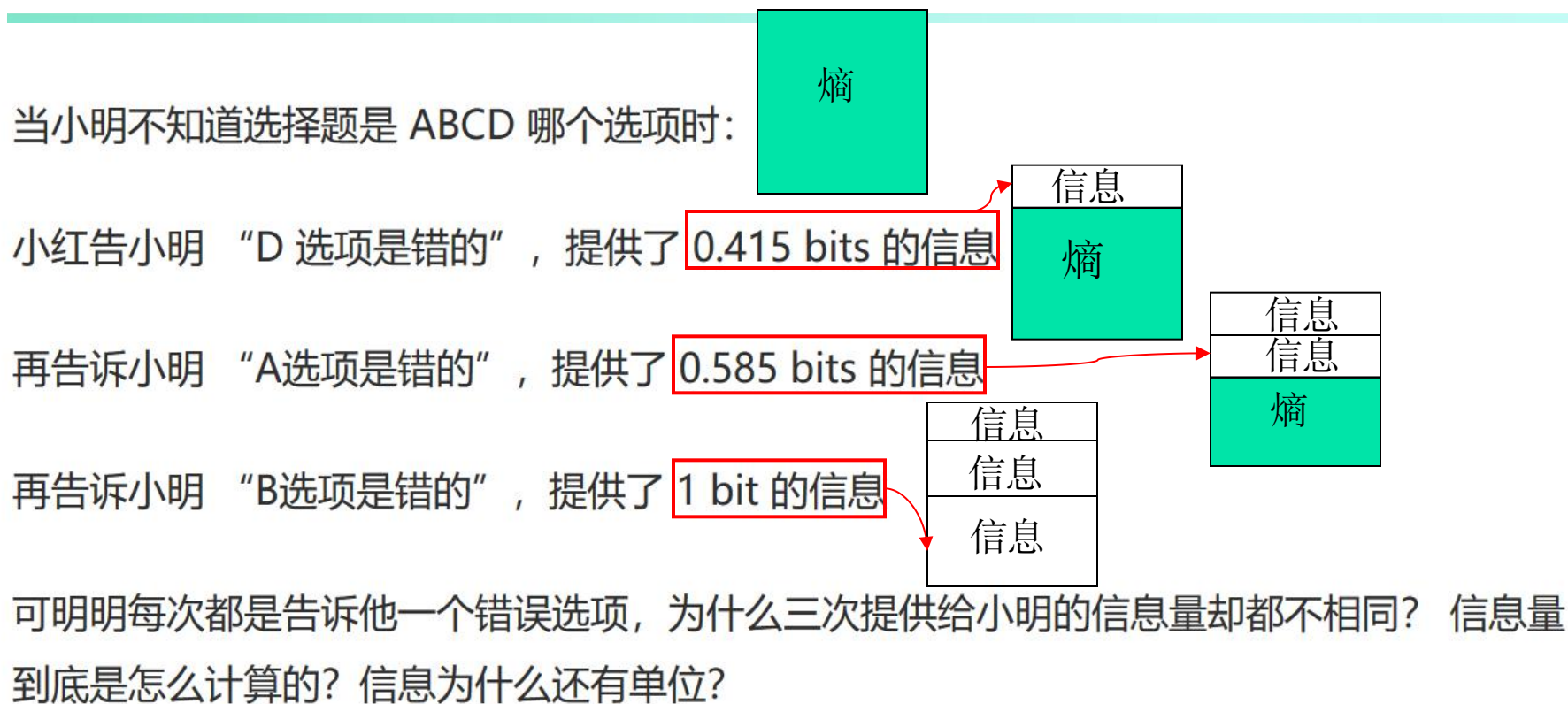
- 1 某个节点上的所有样本都属于相同的类别
- 2 所有属性都用到了 - 采用多数有效法对叶子节点分类
- 3 没有样本了

决策树分类第一步：选择属性，作为树根

- 比较流行的属性选择方法：信息增益
- 信息增益最大的属性被认为是最好的树根

下面先来了解什么是熵，什么是信息，以及如何度量它们

[http://www.360doc.com/content/
19/0610/07/39482793_841453815.shtml](http://www.360doc.com/content/19/0610/07/39482793_841453815.shtml)



回想一下，什么东西有单位？质量，温度等物理量。

没错，信息也是一个物理量。要测量这个物理量，不妨回想一下我们是怎么测量质量的，“千克”最初又是怎么被定义出来的？

其实最初我们并不知道千克的质量，而是选择了一个参照物，把这个物体的质量就称为千克。当想要测量其他物体的质量时，就看这个物体的质量相当于多少个参照物体的质量。这里的“多少个”便是千克。如果换另一个参照物体，那么单位就会变化，比如斤。

测量信息是也是一样，既然信息消除的是不确定性，那么就选择另一个事件的不确定性作为参照事件。当想要测量其他事件的信息时，就看看待测事件的不确定性相当于多少个参照事件的不确定性。这里的“多少个”便是信息量。

当选择的参照事物是像抛硬币这样，只有 2 种等概率情况的事件时，测得的信息量的单位就被称为比特。

有 e (自然底数) 种等概率情况的事件时，测得的信息量的单位就被称为 nat。

有 10 种等概率情况的事件时，测得的信息量的单位就被称为 ban。

然而测量质量时，我们是用 待测物体的质量 除以 参照物体的质量。因为 待测物体的质量 m 等于 参照物体的质量 K 和 乘以 参照物体个数 n ，所以当知道 m 要求 n 时，我们用乘法的反函数，即除法来计算。

可是测量信息时，却不能用除法，因为 抛掷 3 个硬币能够产生的等可能结果并非 $3*2=6$ ，而是 $2^3=8$ 种。也就是说 待测不确定情况的个数 m 是由 参照不确定情况的个数 K 的参照物个数 n 是指数关系进行累积的。

所以当知道可能情况的个数 m ，想求这些情况相当于多少个 n 参照事件所产生的时，我们用指数运算的反函数，即对数运算来计算。

这样 8 个不确定情况就相当于 3 个硬币抛 出的结果， 4 个不确定情况就相当于 抛 2 个硬币得出的，故小明对答案是ABCD里哪一选项的不确定性： $\log_2 4 = 2 \text{ bits}$ 。

但这里有个前提，那就是被测事件的所有可能情况都必须是等概率才行，因为参照事件本身的两种情况就是等概率。

可是当不知道答案的小明被告知“有一半可能性是 C 选项”时，各个情况的概率就不一样了，这时该如何计算熵呢？

答案是分别测量待测事件每种可能情况的信息量后，乘以它们各自的发生概率再相加即可。

不过，怎么测量每种情况的信息量呢？怎么知道概率为 $1/6$ 的情况的不确定性相当于抛掷多少次硬币所产生的不确定性呢？

我们确实没法再用 $\log m$ 这个公式了，但我们知道 1% 会发生的情况，相当于从 100 个等概率情况中确定实际情况，概率的倒数等于等概率情况的个数， $m = 1/p$ 。

用 $1/p$ 替换 m 后，我们就可以计算每种情况的信息量了，再用每个情况的信息量乘以对应发生的概率，再相加后，就能算总信息量了。

$$-\sum_i p_i \log_2 p_i$$

i : 用于指定第几个具体情况

p_i : 第 i 个具体情况的概率

Σ : 表示 i 个情况计算出的 $p \log p$ 累加起来

之所以前面有负号是因为

$$\log 1/p = \log p^{-1} = -\log p$$

概率的倒数变成了负号被拿到前面了

回到例子中，不知道任何答案的小明对 答案的不确定性是：

$$\log_2 1/p = \log_2 \frac{1}{4} = 2$$

当被告知 C 有 50%概率是正确答案时，小明对答案的不确定性是：

$$1/6 * \log_2 6/1 +$$

$$1/6 * \log_2 6/1 +$$

$$1/2 * \log_2 2/1 +$$

$$1/6 * \log_2 \frac{1}{6} = 1.79$$

$2 - 1.79 = 0.21$ bits，也就是小红告诉小明“ C 的概率是 50% ”时提供的信息。

可以思考一下：

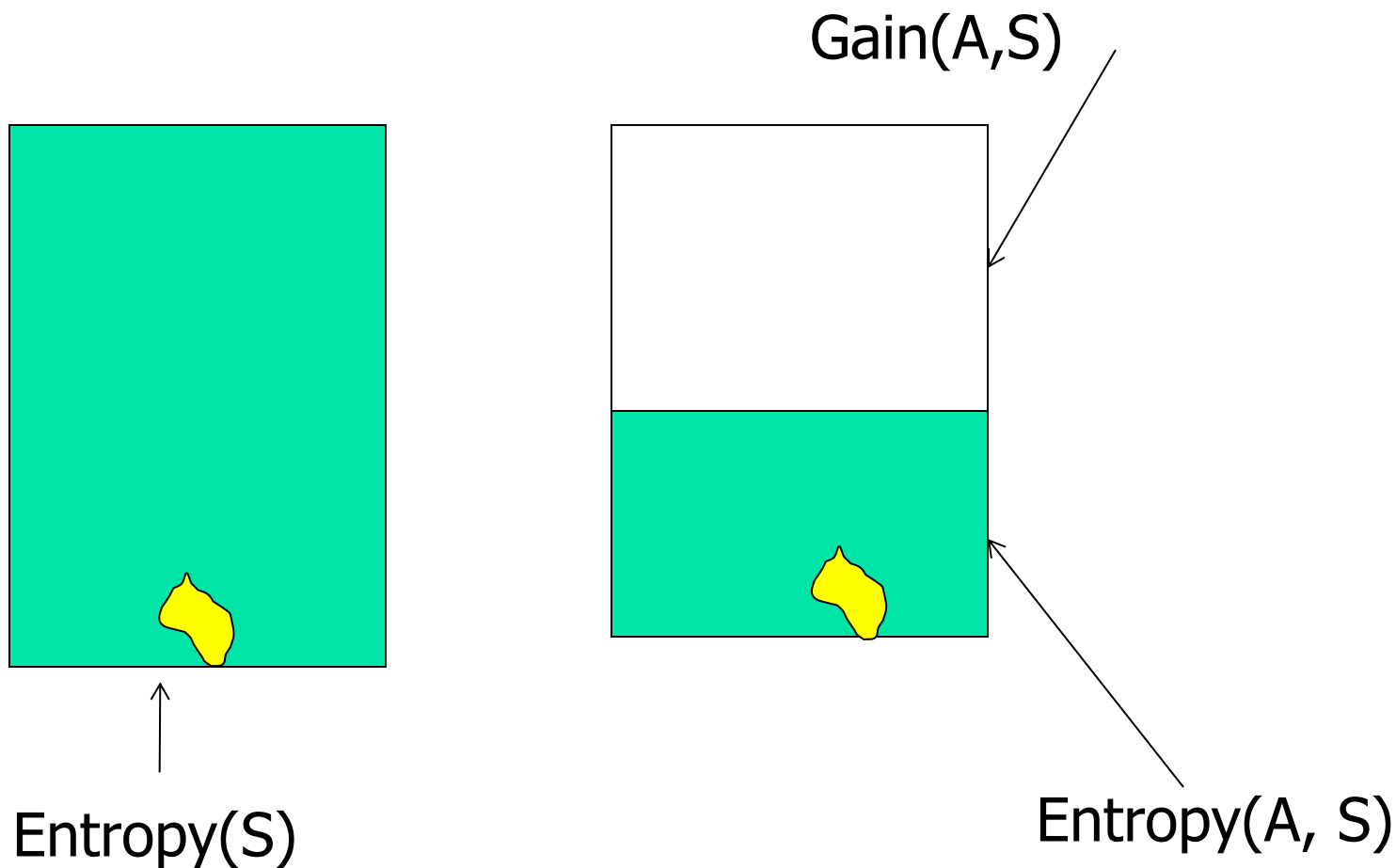
为什么三次提供给小明的信息量却都不相同？

当小明不知道选择题是 ABCD 哪个选项时: 熵= $4 * (-1/4 \log_2 1/4) = 2$

小红告小明 “D 选项是错的”, 提供了 0.415 bits 的信息 熵= $3 * (-1/3 \log_2 1/3) = 1.585$
信息 = $2 - 1.585 = 0.415$

再告诉小明 “A选项是错的”, 提供了 0.585 bits 的信息 熵= $2 * (-1/2 \log_2 1/2) = 1$
信息 = $1.585 - 1 = 0.585$

再告诉小明 “B选项是错的”, 提供了 1 bit 的信息 熵= $-1 \log_2 1 = 0$
信息 = $1 - 0 = 1$



熵表示不确定性的**大小**，**信息**用来消除不确定性

决策树分类第一步：选择属性，作为树根

- 比较流行的属性选择方法：信息增益
- 信息增益最大的属性被认为是最好的树根

实际上，给定训练集 S ，信息增益代表的是在不考虑任何输入变量的情况下确定 S 中任一样本所属类别需要的信息（以消除不确定性）与考虑了某一输入变量 X 后确定 S 中任一样本所属类别需要的信息之间的差。差越大，说明引入输入变量 X 后，消除的不确定性越大，该变量对分类所起的作用就越大，因此被称为是好的分裂变量。换句话说，输入变量 X 为分类这个数据挖掘任务带来了信息增益。信息增益越大，说明输入变量 X 越重要，因此应该被认为是好的分裂变量而优先选择。

决策树分类第一步：选择属性，作为树根

因此，计算信息增益的总的思路是：

- 1) 首先计算不考虑任何输入变量的情况下要确定 S 中任一样本所属类别需要的熵 $\text{Entropy}(S)$;
- 2) 计算引入每个输入变量 X 后要确定 S 中任一样本所属类别需要的熵 $\text{Entropy}(X, S)$;
- 3) 计算二者的差， $\text{Entropy}(S) - \text{Entropy}(X, S)$ ，此即为变量 X 所能带来的信息（增益），记为 $\text{Gain}(X, S)$ 。

- 用S表示训练集，假设分类属性具有m个不同的值，也就是说共有m个不同的分类 $C_i (i = 1, \dots, m)$ ，用 s_i 表示S中属于分类 C_i 的样本的个数

- 则信息收益可以用如下三步求出

- 求总的熵:

$$\text{Entropy}(S) = - \sum_{i=1}^m \frac{s_i}{S} \log_2 \frac{s_i}{S}$$

- 对每个属性，求已知该属性后的熵，假设属性A的值为 $\{a_1, a_2, \dots, a_v\}$

$$\text{Entropy}(A, S) = \sum_{j=1}^v \frac{s_j}{S} \text{Entropy}(S_j)$$

- 对每个属性，求信息增益:

属性A的第j个取值的样本比例, 样本越多越重要

$$\text{Gain}(A, S) = \text{Entropy}(S) - \text{Entropy}(A, S)$$

属性选择方法：信息增益计算示例

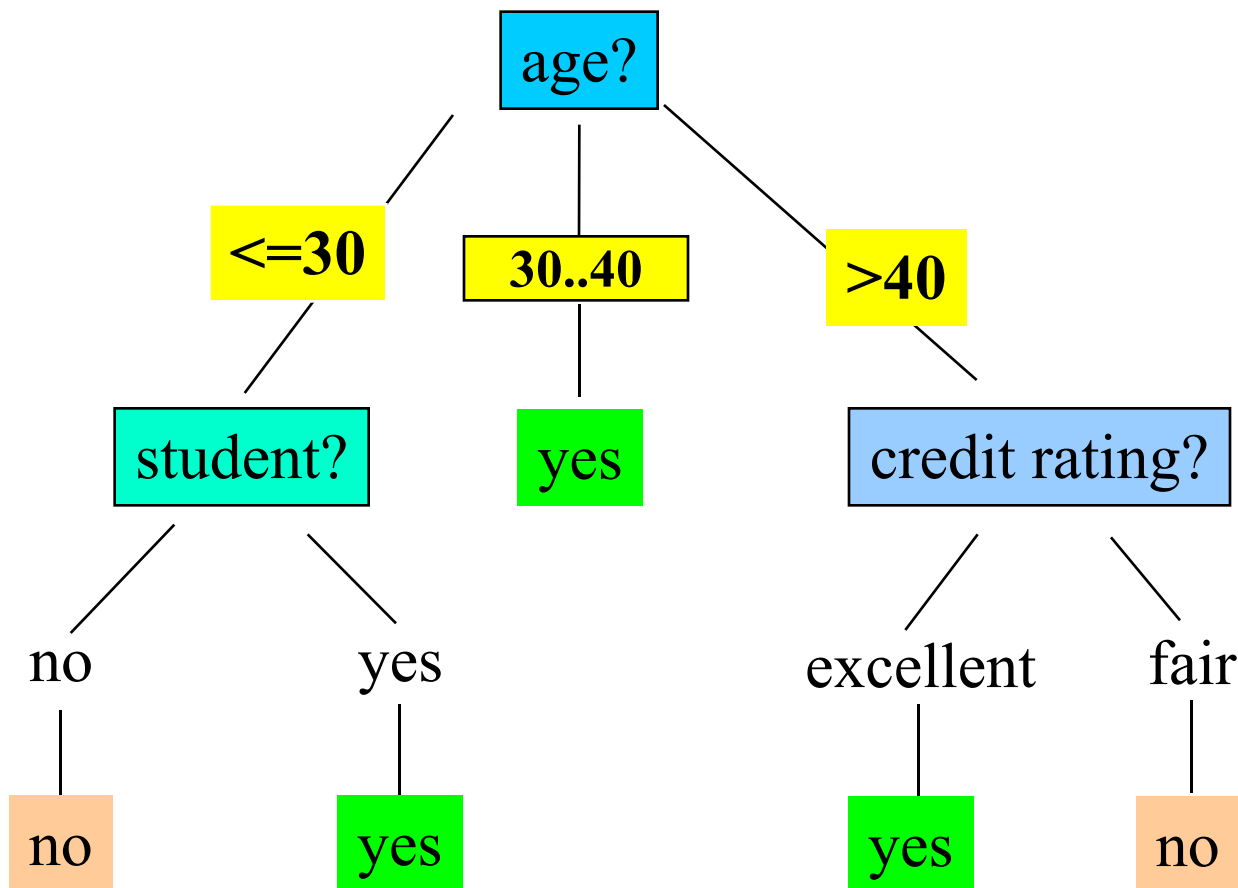
共有5个属性

前4个属性用作
预测属性，最
后一个属性是
类别属性

共有14个样本
，或者说14条
记录

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

属性选择方法：信息增益计算示例



属性选择方法：信息增益计算示例

g Class P: buys_computer = "yes"

g Class N: buys_computer = "no"

g $E(p, n) = E(9, 5) = 0.940$

g Compute the entropy for age $E(\text{age})$:

$$E(p, n) = E\left(\frac{9}{14}, \frac{5}{14}\right) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.940$$

$$E(\text{age}) = \frac{5}{14} E(2, 3) + \frac{4}{14} E(4, 0) + \frac{5}{14} E(3, 2) = 0.694$$

$\frac{5}{14} E(2, 3)$ means "age ≤ 30 " has 5 out of 14 samples, with 2 yes'es and 3 no's. Hence

age	p_i	n_i	$I(p_i, n_i)$
≤ 30	2	3	0.971
30...40	4	0	0
> 40	3	2	0.971

$$\text{Gain}(\text{age}) = E(p, n) - E(\text{age}) = 0.246$$

Similarly,

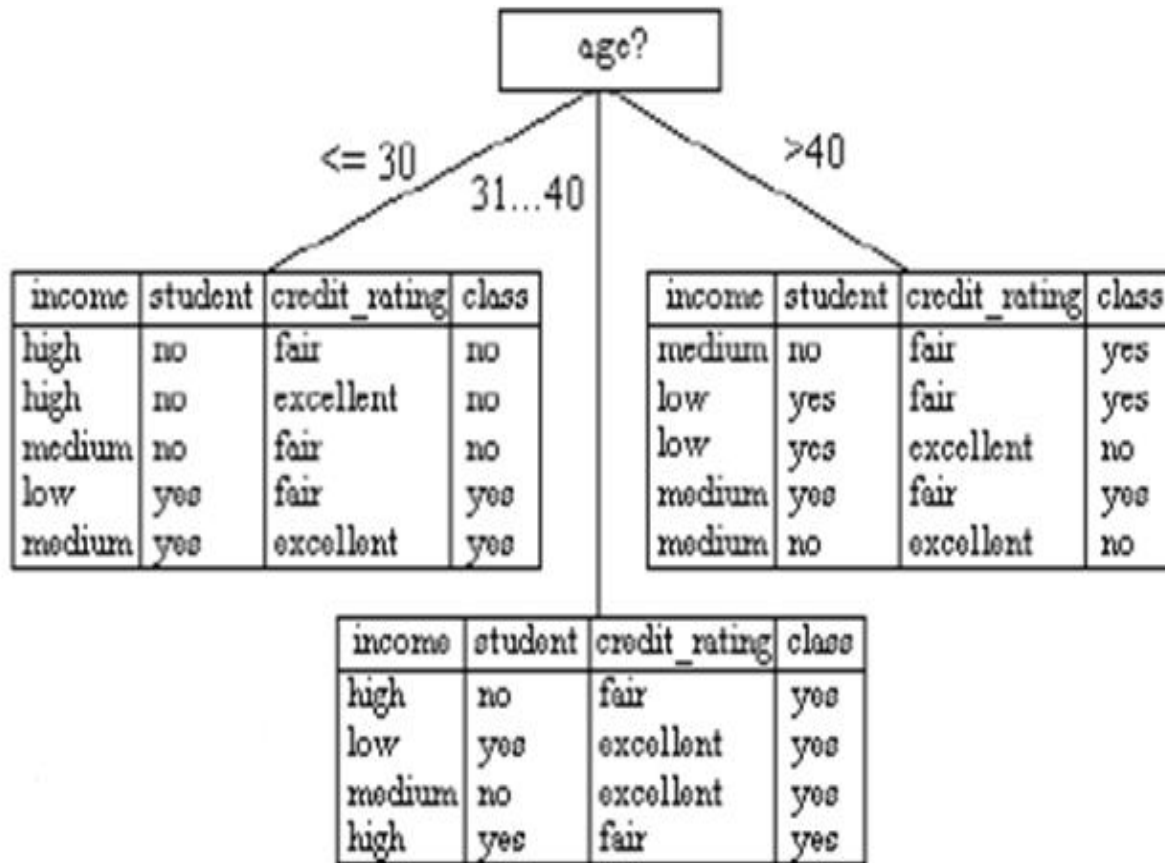
$$\text{Gain}(\text{income}) = 0.029$$

$$\text{Gain}(\text{student}) = 0.151$$

$$\text{Gain}(\text{credit_rating}) = 0.048$$

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31...40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31...40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
> 40	medium	no	excellent	no

根据树根，划分训练集



根据属性 age 进行数据集划分

请计算左、右子树接下来分别该选择哪个属性做树根？

Extracting Classification Rules from Trees

- 决策树中所蕴含的知识可以表达成**IF-THEN**规则的形式
- 从根到叶的一条路径生成一条规则
- 路径上的属性值由**AND**连接起来，构成**IF**部分
- 叶子节点构成**THEN**部分，指出所属的分类
- 这样的规则易于被人们所理解
- Example

IF *age* = "<=30" AND *student* = "no" THEN *buys_computer* = "no"

IF *age* = "<=30" AND *student* = "yes" THEN *buys_computer* = "yes"

IF *age* = "31...40" THEN *buys_computer* = "yes"

IF *age* = ">40" AND *credit_rating* = "excellent" THEN *buys_computer* = "yes"

IF *age* = "<=30" AND *credit_rating* = "fair" THEN *buys_computer* = "no"

Avoid Overfitting in Classification

- **Overfitting:** 有些生成的决策树完全服从于训练集，太循规蹈矩，以至于生成了太多的分支，某些分支可能是一些特殊情况，出现的次数很少，不具有代表性，更有甚者仅在训练集中出现，导致模型的准确性很低
- 通常采用剪枝的方式来克服 **overfitting**，剪枝有两种方法：
 - 先剪：构造树的过程中进行修剪。不符合条件的分支则不建。
 - 后剪：整个树生成之后进行修剪

先剪 vs. 后剪

□ 时间开销:

- 先剪: 训练时间开销降低, 测试时间开销降低
- 后剪: 训练时间开销增加, 测试时间开销降低

□ 过/欠拟合风险:

- 先剪: 过拟合风险降低, 欠拟合风险增加
- 后剪: 过拟合风险降低, 欠拟合风险基本不变

□ 泛化性能: 后剪 通常优于 先剪

西瓜书中**ID3**决策树的实现

https://blog.csdn.net/leafage_m/article/details/79629074

Computing Information-Gain for Continuous-Value Attributes

■ 假设属性A是一个连续值属性

由于连续属性的可取值数目不再有限，因此，不能直接根据连续属性的可取值来对节点进行划分。此时，我们可以使用连续属性离散化的技术将连续属性转换为离散的。

最简单的策略是使用二分法对连续属性进行处理

给定样本集 D 和连续属性 a ，假定 a 在 D 上出现了 n 个不同的取值，将这些值从小到大进行排序，记为 $\{a^1, a^2, a^3, \dots, a^n\}$ 。基于划分点 t 可将 D 分为子集 D_t^- 和 D_t^+ 。其中 D_t^- 包含那些在属性 a 上取值不大于 t 的样本，而 D_t^+ 则包含那些在属性 a 上取值大于 t 的样本。显然，对相邻的属性取值 a^i 与 a^{i+1} 来说， t 在区间 $[a^i, a^{i+1})$ 中取任意值所产生的划分结果相同。因此，对于连续属性 a ，我们可考察包含 $n - 1$ 个元素的候选划分集合。

$$T_a = \left\{ \frac{a^i + a^{i+1}}{2} \mid 1 \leq i \leq n - 1 \right\}$$

即把区间 $[a^i, a^{i+1})$ 的中位点 $\frac{a^i + a^{i+1}}{2}$ 作为候选划分点。然后我们就可像离散属性值一样来考察这些划分点，选取最优的划分点进行样本集合的划分。

利用书中的西瓜数据集3.0作为样本集 D ，连续属性 a 就是其中的密度。那么在数据集中密度一共有17（17也就是概念中的 n ）中不同的取值，将这17个值从小到大进行排序为：

```
1 [0.243, 0.245, 0.343, 0.36, 0.403, 0.437, 0.481, 0.556, 0.593, 0.608, 0.634, 0.639, 0.657, 0.666, 0.697, 0.746]
```

然后利用公式求出 T_a ，其中 T_a 每个元素为： $\frac{a^i + a^{i+1}}{2}$ ，也就是相邻两个数值的平均值，比如0.243和0.245的平均值为：0.244、0.245和0.343的平均值为：0.294。。。按照这种计算方法就可以将密度计算出16个候选值：

```
1 [0.244, 0.294, 0.351, 0.381, 0.420, 0.459, 0.518, 0.574, 0.600, 0.621, 0.636, 0.648, 0.661, 0.708, 0.746]
```

得到了候选值之后我们就把它们看作为离散值，可以利用这些候选值将数据集划分为“大于”和“小于”两个部分。比如说使用0.244可以将数据集划分为：“密度小于0.244”和“密度大于0.244”这两个部分，然后进行接下来的计算。

连续属性离散后的信息增益计算。

上述部分已经使用二分法的技术将连续属性离散化。接下来就需要计算出来离散值后的信息增益了。

将计算信息增益的公式改造为：

$$\text{Gain}(D, a) = \max_{t \in T_a} \text{Gain}(D, a, t) = \max_{t \in T_a} \text{Ent}(D) - \sum_{\lambda \in \{-, +\}} \frac{|D_t^\lambda|}{|D|} \text{Ent}(D_t^\lambda)$$

公式主要由两部分构成，其前半部分的： $\text{Ent}(D)$ 还是当前集合的信息熵，这一点的计算过程与之前的离散属性计算一致。所以这里的值依然为： $\text{Ent}(D) = 0.998$

后半部分的计算与之前的略有不同。后半段公式中使用到了上述得到的候选值，也就是上述的16个值，而且对于每个候选值我们用 λ 将集合分为两部分，一种是小于当前候选值的部分，另一种是大于当前候选值的部分。比如我们取第五个候选值0.420，然后将集合中密度值小于0.420的划分为一部分，将大于0.420的划分为另一部分。

所以小于0.420的密度值有： $\{0.243, 0.245, 0.343, 0.36, 0.403\}$ ，这些密度值对应的样本编号为： $\{10, 11, 12, 15, 6\}$ ，对应的标签为： $\{\text{坏瓜}, \text{坏瓜}, \text{坏瓜}, \text{坏瓜}, \text{好瓜}\}$ 。

所以这部分计算的 $\frac{|D_t^\lambda|}{|D|}$ 的值为： $\frac{5}{17}$ 。

这部分计算的 $Ent(D_t^\lambda)$ 为： $Ent(D_t^\lambda) = -(\frac{1}{5} \times \log_2 \frac{1}{5} + \frac{4}{5} \times \log_2 \frac{4}{5}) = 0.72$

大于0.420的密度值有： $\{0.437, 0.481, 0.556, 0.593, 0.608, 0.634, 0.639, 0.657, 0.666, 0.697, 0.719, 0.774\}$ ，对应的标签为： $\{\text{好瓜}, \text{好瓜}, \text{好瓜}, \text{坏瓜}, \text{好瓜}, \text{好瓜}, \text{坏瓜}, \text{坏瓜}, \text{坏瓜}, \text{好瓜}, \text{坏瓜}, \text{好瓜}\}$ 。

这部分计算的 $\frac{|D_t^\lambda|}{|D|}$ 的值为： $\frac{12}{17}$

这部分计算的 $Ent(D_t^\lambda)$ 为： $Ent(D_t^\lambda) = -(\frac{7}{12} \times \log_2 \frac{7}{12} + \frac{5}{12} \times \log_2 \frac{5}{12}) = 0.980$

所以在0.420这个划分点计算的信息增益为：

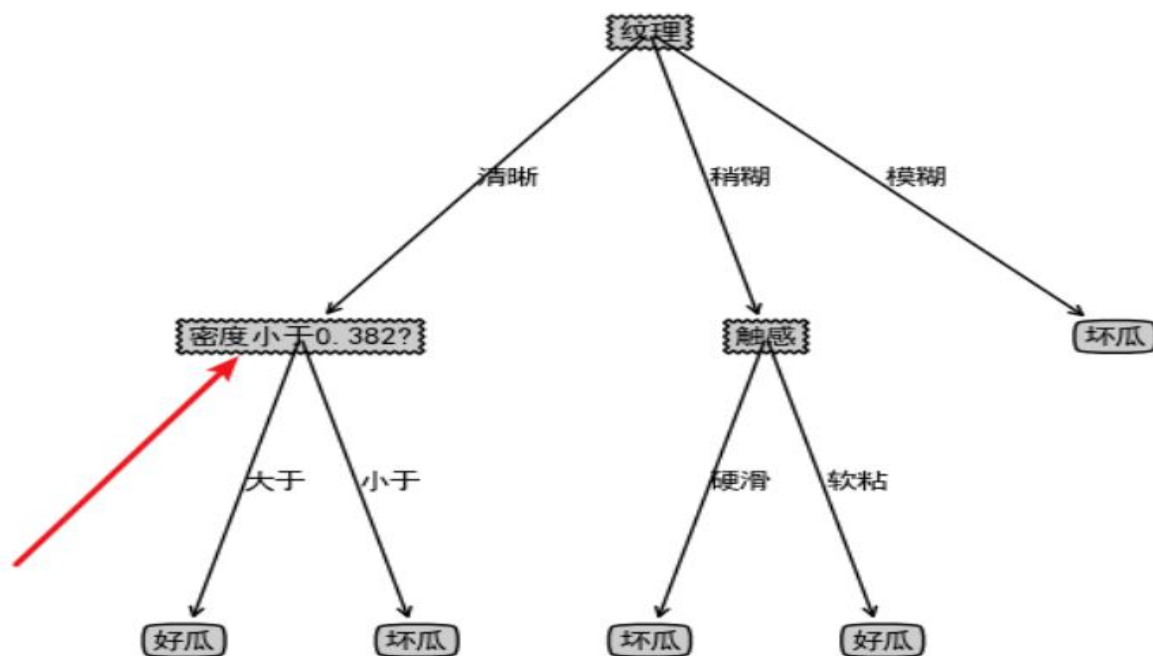
$$Ent(D) - \sum_{\lambda \in \{-, +\}} \frac{|D_t^\lambda|}{|D|} Ent(D_t^\lambda) = 0.998 - (\frac{5}{17} \times 0.72 + \frac{12}{17} \times 0.980) = 0.094$$

这样我们就通过计算得到了在候选划分点0.420的信息增益，同时在公式的前面还有一个 $\max(t \in T_a)$ ，也就是说我们需要计算出所有候选划分点 T_a 的各个信息增益，然后取其中的最大最为该密度属性的信息增益。

创建决策树

首先，如果是连续属性的话，比如说“密度”，我们计算出信息增益之后，我们需要得到该信息增益对应的划分点，然后将集合划分为“小于。。。 ”和“大于。。。 ”两部分，所以在决策树该划分点的左右两部分分别就是这两个子集。

其次，与离散值不同，若当前划分属性为连续属性，该属性还可作为其后代节点的划分属性。比如在父节点上使用了“密度小于等于0.381”，不会禁止在子节点上使用“密度小于等于0.294”的。



信息增益的理解

- 对于待划分的数据集D，其 $\text{entropy}(\text{前})$ 是一定的，但是划分之后的熵 $\text{entropy}(\text{后})$ 是不定的， $\text{entropy}(\text{后})$ 越小说明使用此特征划分得到的子集的不确定性越小（也就是纯度越高），因此 $\text{entropy}(\text{前}) - \text{entropy}(\text{后})$ 差异越大，说明使用当前特征划分数数据集D的话，其纯度上升的更快。在构建最优决策树的时候，总希望能更快速到达纯度更高的集合，因此总是选择使得信息增益最大的特征来划分当前数据集D
- 缺点：信息增益偏向取值较多的特征
 - 原因：当特征的取值较多时，根据此特征划分更容易得到纯度更高的子集，因此划分之后的熵更低，由于划分前的熵是一定的，因此信息增益更大，因此信息增益比较偏向取值较多的特征

Gain Ratio for Attribute Selection (C4.5)

- 为了解决这个问题，C4.5 (ID3的改进) 采用 gain ratio 作为度量手段

- $\text{GainRatio}(A) = \text{Gain}(A) / \text{SplitInfo}(A)$

$$\text{SplitInfo}_A(D) = - \sum_{j=1}^v \frac{|D_j|}{|D|} \times \log_2 \left(\frac{|D_j|}{|D|} \right)$$

- 取值个数多的属性，SplitInfo的值大

- **Ex.** $\text{SplitInfo}_A(D) = -\frac{4}{14} \times \log_2 \left(\frac{4}{14} \right) - \frac{6}{14} \times \log_2 \left(\frac{6}{14} \right) - \frac{4}{14} \times \log_2 \left(\frac{4}{14} \right) = 0.926$

- 这里A为income, $\text{gain_ratio}(\text{income}) = 0.029 / 0.926 = 0.031$

- 具有最大的gain ratio的属性被选为划分属性

Classification in Large Databases

- **Classification**—是一个老问题，统计和机器学习的研究人员已经对其进行了广泛的研究
- **Scalability**: 用比较合理的速度对具有上百万样本和上百个属性的数据集进行分类
- 机器学习领域为什么采用决策树呢?
 - 和其他分类方法相比，决策树的学习速度较快
 - 可以转换成简单的、易于理解的分类规则
 - 可以采用**SQL**查询访问数据库
 - 具有较高的分类准确性

决策树的模型解读

数据挖掘的两种建模形式

- 预测建模
- 描述建模

预测建模

- 预测建模的目的则是根据观察到的对象特征值预测它的其他特征值。这里的特征有时也称为变量或属性

预测模型：分类

- 在预测模型中，一个变量被表达成其他变量的函数
- 可以把预测建模的过程看作是学习一种映射或函数 $Y=f(X;\theta)$ 。其中：
 - f 是模型结构的函数形式
 - θ 是 f 中的未知参数。
 - X 通常被称为输入变量，是一个 p 维向量，代表观察到的对象的 p 个属性值。
 - Y 通常被称为响应变量，是一个标量，代表预测的结果

预测模型：分类

- 在函数 $Y=f(X;\theta)$ 中：
 - 如果 Y 是数量型变量，那么学习从向量 X 到 Y 的映射的过程叫做回归
 - 如果 Y 是范畴型变量，则叫做分类

预测模型

- 从学习一个 p 维向量 X 到 Y 的映射这个角度来讲, 分类和回归这两种任务都可以看作是函数逼近(function approximation)问题。

预测建模的训练数据

- 由 n 对 (X, Y) 组成。每对数据中的向量 $X(i)$ 和目标值 $Y(i)$ 都是从已知数据中观察得到的($0 \leq i \leq n$)。
- 预测建模就是：根据训练数据拟合出模型 $Y = f(X; \theta)$ ，该模型可以在给定输入向量 X 和模型 f 的参数 θ 的情况下预测出 Y 的值。

预测模型的拟合过程

■ 需要完成以下事情：

- 1) 确定模型 f 的结构；
- 2) 确定参数 θ 的值。 θ 值是通过在数据集上最小化（或最大化）一个评分函数来确定的，而搜索最佳 θ 值的过程就是优化的过程，通常是数据挖掘算法的核心部分。

因此，从算法组件的角度出发，模型拟合的过程实际上也就是要确定模型结构、评分函数以及搜索优化策略。

用于预测的模型结构

- 由于事先不知道模型 $f(\mathbf{X};\boldsymbol{\theta})$ 的形式，为 f 本身选择一个合适的函数形式本身是非常具有挑战性的。
- 回归模型和分类模型都建立在很多相同的数学和统计基础之上，用于其中一种任务的模型通常也可以应用于另一种任务。

用于分类的预测模型

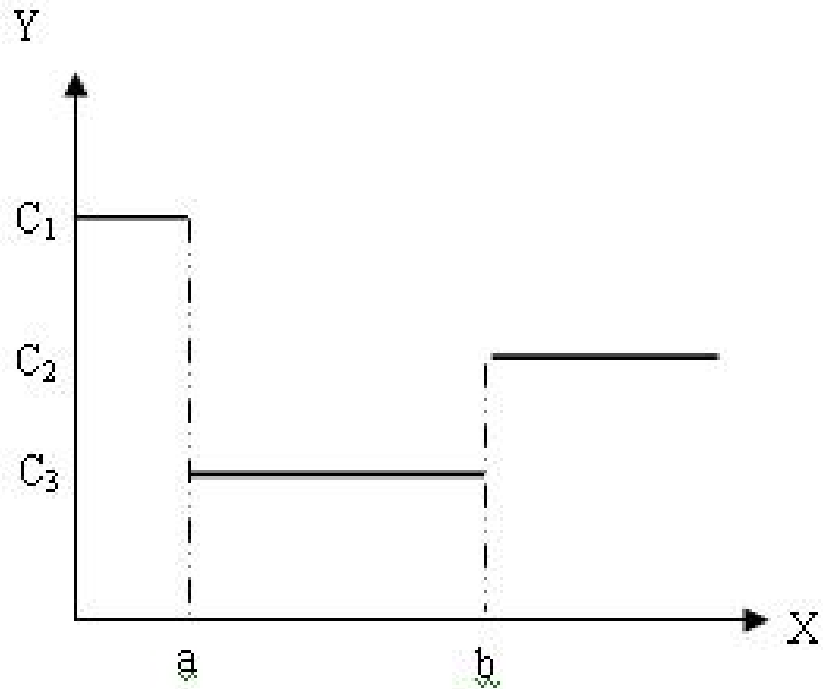
- 判别模型
- 概率模型

判别模型

- 判别模型的输入是输入向量 X ，输出是响应变量 Y 。 Y 的取值为 $\{C_1, C_2, \dots, C_m\}$ ，其中 C_i 表示类别。

判别模型

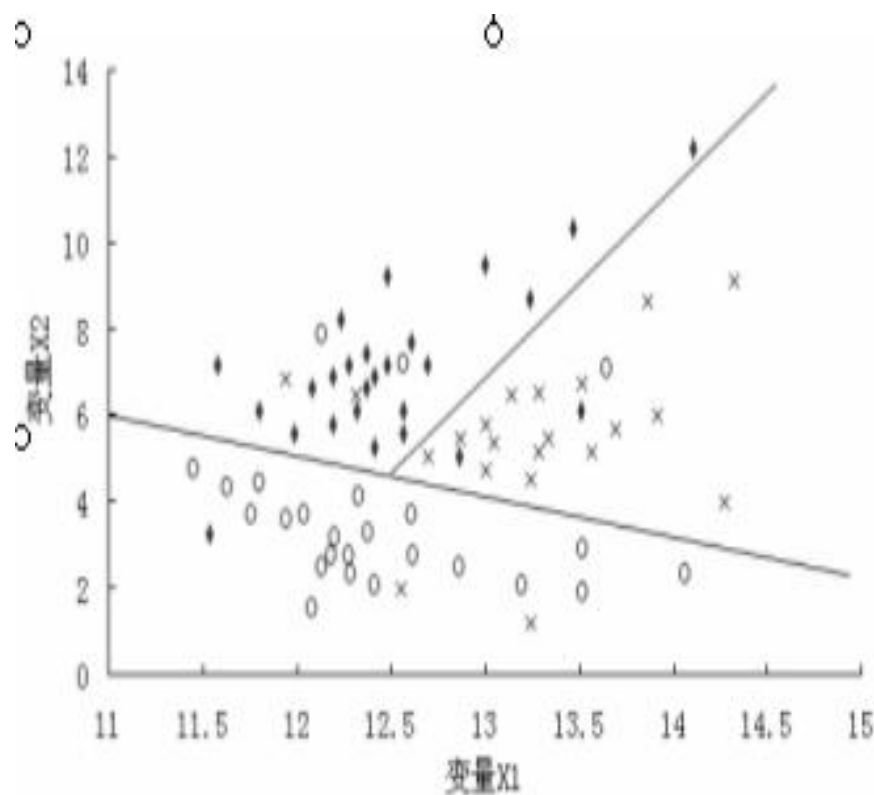
- 例如，当维数 $p=1$ 时，判别模型实际上是二维空间中的分段直线。在一定的区域内，直线的取值为 $C_i (0 \leq i \leq m)$ 。



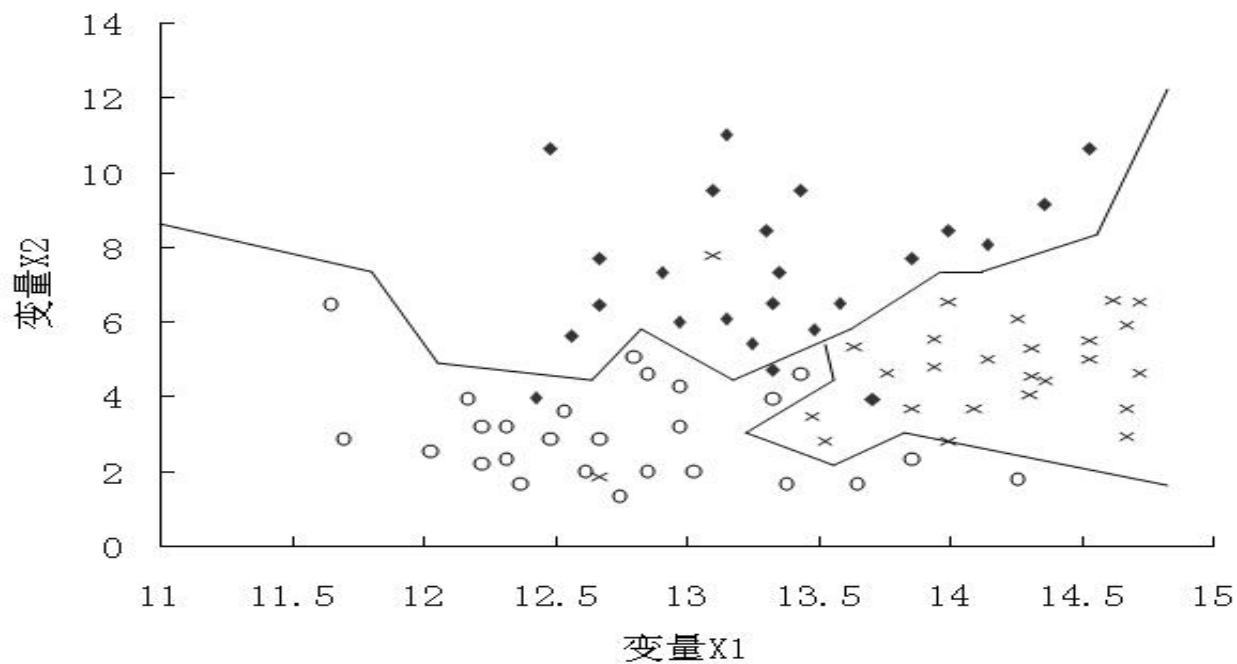
-
- 当维数 $p=2$ 时，判别模型实际上是三维空间中的一个分段曲面。
 - 仅当输入变量 X 的分量 X_1 和分量 X_2 共同构成的平面 (X_1, X_2) 位于一定区域时，该曲面的取值为 $C_i (0 \leq i \leq m)$ 。
 - 取值为 C_i 的所有区域的联合称为 C_i 类的决策区域。意思是，只要输入变量 X 落入这个区域，它的类别就被预测为 C_i 。

-
- 在判别模型中，分类的主要任务是要确定各个类别的决策区域
 - 或者说，我们所感兴趣的是不同类别之间的边界。

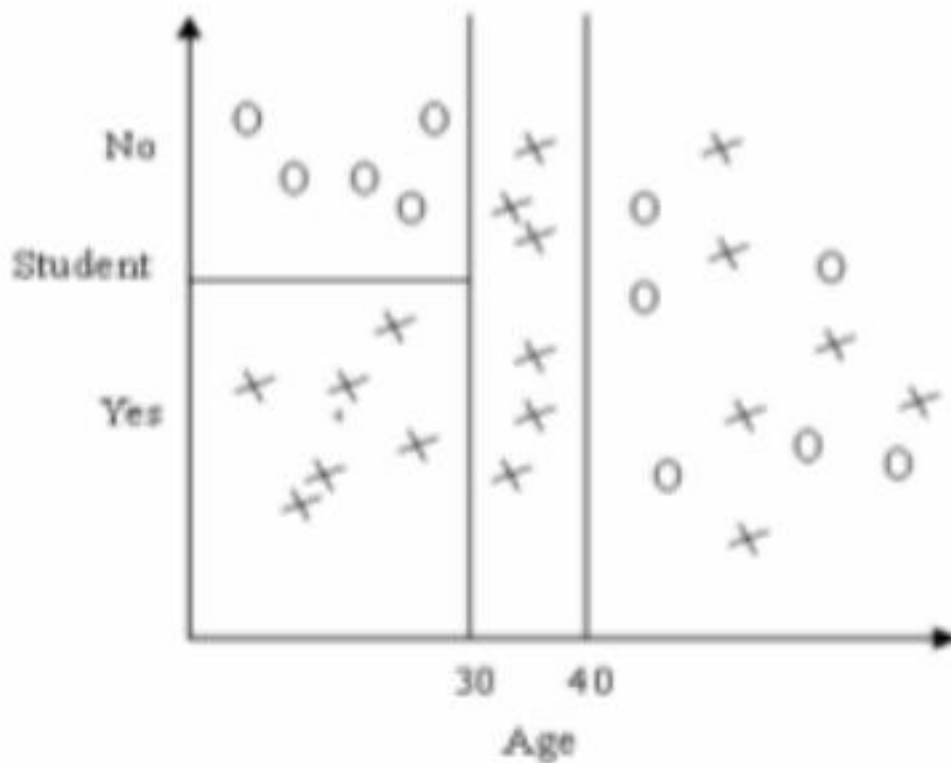
- 和回归的情况类似，可以对类别间边界的函数形式做一个简单的假定。
- 例如，可以用线性边界将X空间分割成不相交的决策区域，每个区域对应一个类别。



- 也可以将线性决策边界分段组合起来



决策树的判别模型



概率模型

- 分类的概率建模是要针对每一个类别 C_i ，估计一种分布或密度函数 $p(X|C_i, \theta_i)$ ，其中 θ_i 是该函数的参数，它反映了 C_i 类的主要特征。

概率模型

- 例如，对于多变量的实数值数据，可以假定每个类别的模型结构都是多元正态分布，而且参数 θ_i 代表每个类的均值（位置）和方差（范围）特征。
- 如果各个均值离得足够远，而且方差足够小，则各个类在输入空间中可以被很好地分割开来，从而使分类的准确性最高

用于预测的评分函数

- 对于分类，普遍使用的是误分类率
- 对于回归，普遍使用的评分函数是误差平方和